1- Project Title

Distributed Smart Security Surveillance System using ROS

2- Project Overview

You must design a **distributed smart surveillance system** using ROS.

The system analyzes a camera stream or video file to detect objects and monitor suspicious activity.

The system integrates:

- camera streaming
- object detection using **YOLO**
- depth estimation using **DepthAnythingV2**
- event detection
- security response coordination

The system consists of **8 nodes** working together.

Input sources:

• Laptop camera
• Video files

No additional sensors or hardware are required.

3- Core Idea

The system continuously analyzes a video stream.

The pipeline is:

1 Camera captures frames
2 YOLO detects objects
3 Depth estimation estimates distance
4 Scene analyzer evaluates environment

5 Event manager identifies suspicious events
6 Security controller triggers responses
7 Logger records events
8 Monitor displays system status

All nodes communicate using ROS.

4- Required Nodes

4.1 Camera Stream Node

Responsibility

Capture frames from:

- laptop camera
- video file

Topics Published

`/camera_frames`

Topics Subscribed

None

Services Provided

None

Services Used

None

Actions Used

None

Parameters Used

- `camera_source`
- `frame_rate`

Internal Logic

Continuously capture frames and publish them.

4.2 Object Detection Node

Responsibility

Detect objects using **YOLO**.

Topics Published

`/detected_objects`

Topics Subscribed

`/camera_frames`

Services Provided

None

Services Used

None

Actions Used

None

Parameters Used

- `confidence_threshold`
- `model_path`

Internal Logic

Run YOLO on each frame and publish bounding boxes.

4.3 Depth Estimation Node

Responsibility

Estimate object distance using **DepthAnythingV2**.

Topics Published

`/object_depth`

Topics Subscribed

`/camera_frames`

Services Provided

None

Services Used

None

Actions Used

None

Parameters Used

- `depth_model_path`

Internal Logic

Generate depth map and estimate object distance.

4.4 Scene Analysis Node

Responsibility

Combine detection and depth information.

Topics Published

```
/scene_analysis
```

Topics Subscribed

```
/detected_objects
/object_depth
```

Services Provided

None

Services Used

None

Actions Used

None

Parameters Used

- `danger_distance`

Internal Logic

Identify objects that are too close or unusual.

4.5 Event Manager Node

Responsibility

Detect security events.

Topics Published

```
/security_event
```

Topics Subscribed

```
/scene_analysis
```

Services Provided

None

Services Used

None

Actions Used

None

Parameters Used

- `restricted_objects`

Internal Logic

Identify events like:

- person in restricted area
- object too close to camera

4.6 Security Response Node

Responsibility

Trigger response actions.

Topics Published

`/security_alert`

Topics Subscribed

`/security_event`

Services Provided

None

Services Used

None

Actions Used

`/security_action`

Parameters Used

- `alert_level`

Internal Logic

Launch long-running response actions.

4.7 Event Logger Node

Responsibility

Record system activity.

Topics Subscribed

```
/security_event
/security_alert
```

Internal Logic

Store events and print logs.

4.8 System Monitor Node

Responsibility

Observe system health.

Topics Subscribed

All main topics.

Internal Logic

Display system status and performance.

5- System Rules

Allowed inputs:

- webcam
- video files

Detection frequency:

minimum **5 FPS**

Depth estimation must run continuously.

System must detect at least **two object types**.

6- Required ROS Communication

Topics

Topic	Purpose
/camera_frames	camera images
/detected_objects	YOLO detection
/object_depth	depth estimation
/scene_analysis	combined analysis
/security_event	detected events
/security_alert	triggered alerts

Services

Optional service may be used for configuration.

Actions

`/security_action`

Used for long security responses.

Parameters

Examples:

- detection confidence
- depth threshold
- event thresholds

7- Main Objectives

Students must implement:

A. Vision Pipeline

Process camera frames through multiple nodes.

B. Multi-Node Coordination

Detection, depth, and decision nodes must cooperate.

C. Action Communication

Security responses must use ROS actions.

D. Parameter Configuration

System behavior must be adjustable using ROS parameters.

E. Debugging Distributed Systems

Students must ensure reliable message flow.

8- Team Division (Mandatory)

Student	Node
Student 1	Camera Stream
Student 2	Object Detection
Student 3	Depth Estimation
Student 4	Scene Analysis
Student 5	Event Manager
Student 6	Security Response
Student 7	Event Logger
Student 8	System Monitor

9- Execution Method

Run nodes in separate terminals.

ROS1

```
roscore

rosrun your_package camera_stream
rosrun your_package object_detector
rosrun your_package depth_estimator
rosrun your_package scene_analyzer
rosrun your_package event_manager
rosrun your_package security_response
rosrun your_package event_logger
rosrun your_package system_monitor
```

ROS2

```
ros2 run your_package camera_stream
ros2 run your_package object_detector
ros2 run your_package depth_estimator
ros2 run your_package scene_analyzer
ros2 run your_package event_manager
ros2 run your_package security_response
ros2 run your_package event_logger
ros2 run your_package system_monitor
```

10- Submission Requirements

Teams must submit:

1 Source code
2 Running demonstration video + The achieved output
3 Explanation of each node

Each student must explain:

- node logic
- ROS communication used
- system integration

Students must also mention **ROS1 vs ROS2 differences**.